B

The Example Tables

This appendix outlines the tables in this book and their use.

Writing SQL statements requires a good understanding of the underlying database design. If you don't know what information is stored in what table, how tables are related to each other, and the actual breakdown of data within a row, it is impossible to write effective SQL.

You are strongly advised to actually try every example in every chapter in this book. All the chapters use a common set of data files. To assist you in better understanding the examples and to enable you to follow along with the chapters, this appendix describes the tables used, their relationships, and how to obtain them.

Understanding the Example Tables

The tables used throughout this book are part of an order entry system used by an imaginary distributor of paraphernalia that might be needed by your favorite cartoon characters (yes, cartoon characters; learning MySQL doesn't need to be boring). The tables are used to perform several tasks:

- Manage vendors
- Manage product catalogs
- Manage customer lists
- Enter customer orders

Making this all work requires six tables that are closely interconnected as part of a relational database design. The following sections describe these six tables.

Note

Simplified Examples The tables used here are by no means complete. A real-world order entry system would have to keep track of lots of other data that has not been included here (for example, payment and accounting information, shipment tracking). However, these tables demonstrate the kinds of data organization and relationships you will encounter in most real installations. You can apply these techniques and technologies to your own databases.

Table Descriptions

The following sections describe the six tables used in this book, as well as the columns in each one.

Note

Why the Order ? If you are wondering why the six tables are listed in the order they are, it is due to their dependencies. The `products` table is dependent on the `vendors` table, so `vendors` is listed first, and so on.

The `vendors` Table

The `vendors` table stores data on the vendors whose products are sold. Every vendor has a record in this table, and the `vend_id` column is used to match products with vendors.

TABLE B.1 `vendors` Table Columns

Column	Description
`vend_id`	Unique numeric vendor ID
`vend_name`	Vendor name
`vend_address`	Vendor address
`vend_city`	Vendor city
`vend_state`	Vendor state
`vend_zip`	Vendor zip code
`vend_country`	Vendor country

Every table should have primary keys defined. This table, for example, should use `vend_id` as its primary key. `vend_id` is an automatically incremented field.

The `products` Table

The `products` table contains the product catalog, with one product per row. Each product has a unique ID (in the `prod_id` column) and is related to its vendor by `vend_id` (the vendor's unique ID).

TABLE B.2 `products` Table Columns

Column	Description
`prod_id`	Unique product ID
`vend_id`	Product vendor ID (which relates to `vend_id` in the `vendors` table)
`prod_name`	Product name

Column	Description
prod_price	Product price
prod_desc	Product description

Every table should have primary keys defined. This table, for example, should use `prod_id` as its primary key.

To enforce referential integrity, a foreign key should be defined on `vend_id`, relating it to `vend_id` in `vendors`.

The `customers` Table

The `customers` table stores all customer information. Each customer has a unique ID (in the `cust_id` column).

TABLE B.3 `customers` Table Columns

Column	Description
cust_id	Unique numeric customer ID
cust_name	Customer name
cust_address	Customer address
cust_city	Customer city
cust_state	Customer state
cust_zip	Customer zip code
cust_country	Customer country
cust_contact	Customer contact name
cust_email	Customer contact email address

Every table should have primary keys defined. This table, for example, should use `cust_id` as its primary key. `cust_id` is an automatically incrementing field.

The `orders` Table

The `orders` table stores customer orders (but not order details). Each order is uniquely numbered (in the `order_num` column). Orders are associated with the appropriate customers by the `cust_id` column (which relates to the customer's unique ID in the `customers` table).

TABLE B.4 `orders` Table Columns

Column	Description
order_num	Unique order number
order_date	Order date
cust_id	Order customer ID (which relates to `cust_id` in the `customers` table)

Every table should have primary keys defined. This table, for example, should use `order_num` as its primary key. `order_num` is an automatically incrementing field.

To enforce referential integrity, a foreign key should be defined on `cust_id`, relating it to `cust_id` in `customers`.

The `orderitems` Table

The `orderitems` table stores the items included in each order, one row per item per order. For every row in `orders`, there are one or more rows in `orderitems`. Each order item is uniquely identified by the order number plus the order item (first item in order, second item in order, and so on). Order items are associated with their appropriate order by the `order_num` column (which relates to the order's unique ID in `orders`). In addition, each order item contains the product ID of the item (which relates the item to the `products` table).

TABLE B.5 `orderitems` Table Columns

Column	Description
`order_num`	Order number (which relates to `order_num` in the `orders` table)
`order_item`	Order item number (sequential within an order)
`prod_id`	Product ID (which relates to `prod_id` in the `products` table)
`quantity`	Item quantity
`item_price`	Item price

Every table should have primary keys defined. This table, for example, should use `order_num` and `order_item` as its primary keys.

To enforce referential integrity, foreign keys should be defined on `order_num`, relating it to `order_num` in `orders`, and `prod_id`, relating it to `prod_id` in `products`.

The `productnotes` Table

The `productnotes` table stores notes associated with specific products. A product may have no associated notes, or it may have many associated notes.

TABLE B.6 `productnotes` Table Columns

Column	Description
`note_id`	Unique `note id`
`prod_id`	Product ID (which corresponds to `prod_id` in the `products` table)
`note_date`	Date the note was added
`note_text`	Note text

Every table should have primary keys defined. This table, for example, should use `note_id` as its primary key.

The column `note_text` must be indexed for `FULLTEXT` search use.

This table uses full-text searching, so `ENGINE=MyISAM` must be specified.

Creating the Sample Tables

To follow along with the examples in this book, you need a set of populated tables. You can find everything you need to get up and running on this book's web page, at http://www.forta.com/books/9780138223021/.

There are two ways to create the sample tables:

- The simplest way is to use MySQL Data Import. This is a simple interactive process that will create the database and fully populate it (in much the same way you'd backup and restore a database).
- You can create the database manually and then run two SQL scripts to first create and then populate the tables.

Both options are described below. Do not use both; use one or the other. The first option is recommended if you are using MySQL Workbench.

Using Data Import

Note

For MySQL 8 Only The data export files used here are intended for use with MySQL 8 and have not been tested with earlier versions of MySQL.

MySQL enables entire databases to be imported and exported. You can download an export file from the book web page and simply import it as follows:

1. Download the data export file and save it somewhere on your computer.
2. In MySQL Workbench, click on the **Administration** tab in the **Navigator** panel on the left.
3. Select **Data Import/Restore**.
4. When the Data Import screen is displayed, select **Import from Self-Contained File** and then select the data export file you saved in step 1.
5. Click the **Start Import** button to create and fully populate the `crashcourse` database.

The new `crashcourse` database should now be displayed in the Schemas tab in the Navigator panel.

Using SQL Scripts

Note

For MySQL Only The scripts and files used here to create the sample tables are very DBMS specific; they are designed to be used only with MySQL.

The scripts have been tested extensively with MySQL 4.1 through MySQL 8 but have not been tested with earlier versions of MySQL.

The book's web page contains two SQL script files that you can download:

- `create.sql` contains the MySQL statements to create the six database tables (and define all primary keys and foreign key constraints).
- `populate.sql` contains the SQL `INSERT` statements used to populate these tables.

After you have downloaded the scripts, you can use them to create and populate the tables needed to follow along with the chapters in this book. Here are the steps to follow:

1. Create a new datasource. (Do not use any existing datasource, just to be on the safe side.) The simplest way to do this is by using MySQL Workbench (described in Chapter 2, "Introducing MySQL").
2. Either click the **Create a new schema** button (fourth from the left, with a picture of a cylinder and a + on it) or select the **Schemas** tab in **Navigator**, right-click, and select **Create Schema**.
3. Name the new schema `crashcourse`. You can ignore all other fields. Click **Apply** to create the new database.
4. Either double-click the new datasource if you're using MySQL Workbench (it'll appear in bold if in use) or use the `USE` command if using the `mysql` command-line utility.
5. Execute the `create.sql` script by using either MySQL Workbench or `mysql`. If you are using MySQL Workbench, select **File**, **Open SQL Script**, **create.sql** and then click the **Execute** button (which has a yellow lightning bolt on it). If using the `mysql` command-line utility, specify `create.sql` as the source by specifying the full path to the `create.sql` file.
6. Execute the `populate.sql` script by using either MySQL Workbench or `mysql`. Use the same procedure as in step 5 to populate the new tables.

Note

Create and Then Populate You must run the table creation scripts *before* you run the table population scripts. Be sure to check for any error messages returned by these scripts. If the creation scripts fail, you will need to remedy whatever problem exists before continuing with table population.

And now you should be good to go!